OPTIMISATION DE MODELES NEURONAUX A OSCILLATEURS COUPLÉS - RAPPORT FINAL

Da Silva Tristan - Battaglia Charles-Antoine
Botté-Magalhaes Jules - Gauthier Keanu - Fraysse Ianis

JUNIA - ISEN - AP4

`https://github.com/CharlesBta/Modele-Neuronaux-RD.git`

Lexique

Backend	Moteur d'exécution d'un simulateur ; Brian2CUDA ajoute par exemple un backend **GPU** au simulateur Brian.
Benchmark	Test de performance comparant les temps d'exécution entre deux architectures, ici CPU et GPU.
C++	Langage de programmation compilé, proche de la machine, utilisé notamment avec **CUDA** pour écrire les programmes rapides.
Canaux ioniques	Pores membranaires permettant le passage sélectif d'ions (sodium, potassium) à travers la membrane du neurone.
Connectome	Carte complète des connexions synaptiques entre les neurones d'un réseau.
CPU	*Central Processing Unit*, processeur généraliste traitant les instructions de façon séquentielle.
CUDA	*Compute Unified Device Architecture* : plateforme de **NVIDIA** pour écrire des programmes qui s'exécutent sur les **GPU**.
Diagramme pieuvre	Schéma qui place le système au centre de ses éléments extérieurs et trace les fonctions qui les relient.
EDO	Équation Différentielle Ordinaire, équation décrivant l'évolution d'une variable par rapport au temps.
Fonction contrainte (FC)	Limite imposée au système par un élément extérieur ; elle restreint la liberté de conception.
Fonction principale (FP)	Service que le système rend à l'utilisateur, raison d'être du système.
Fonction secondaire (FS)	Service qui complète ou rend possible une fonction principale.
Format creux	Stockage d'une matrice qui ne garde que ses éléments non nuls (par exemple **CSR**), adapté aux connectomes peu denses.
FPGA	*Field Programmable Gate Array* : puce reprogrammable dont le matériel s'adapte au calcul, alternative au **GPU** pour les calculs répétitifs.
Génération de code	Production automatique du code **GPU** à partir d'une description de modèle écrite en langage de haut niveau (**Python** ou **Julia**).
GPU	*Graphics Processing Unit*, processeur massivement parallèle conçu pour effectuer un grand nombre de calculs simultanément.
Hodgkin-Huxley	Modèle biophysique décrivant l'évolution du potentiel membranaire d'un neurone à partir de canaux ioniques sodium et potassium.
JAX	Bibliothèque **Python** de calcul différentiable avec vectorisation automatique (vmap).
Jonction communicante	*Gap junction* : couplage électrique direct et bidirectionnel entre deux cellules voisines.
Julia	Langage de programmation scientifique haut niveau, proche du **C++** en performance.
MPI	*Message Passing Interface* : norme d'échange de messages entre machines, utilisée pour répartir une simulation sur plusieurs nœuds de calcul.
NEST	Simulateur de référence pour les grandes populations de neurones simplifiés.
NVLink	Liaison matérielle à très haut débit entre **GPU NVIDIA**.

POC	*Proof of Concept* : preuve de concept, version réduite du système qui démontre la faisabilité de ses fonctions essentielles.
Potentiel membranaire	Différence de potentiel électrique entre l'intérieur et l'extérieur de la membrane d'un neurone.
Python	Langage de programmation interprété, utilisé par le socle de code fourni (solveur de référence, générateurs de connectomes).
PyTorch	Bibliothèque **Python** d'apprentissage automatique, aussi utilisée pour du calcul **GPU** généraliste.
Référence CPU	Implémentation sur **CPU** qui sert d'étalon de justesse et de temps pour calculer le speedup.
RTX 5090	Carte graphique haut de gamme de NVIDIA, équipée d'environ 21 760 cœurs de calcul parallèle.
Speedup	Rapport entre le temps d'exécution sur CPU et sur GPU, mesurant le gain apporté par le GPU.
Synapse	Jonction entre deux neurones assurant la transmission du signal électrique d'un neurone à un autre.
Tri croisé	Méthode de hiérarchisation qui compare les fonctions deux à deux et cumule des points d'importance.
vmap	Opération de vectorisation automatique de **JAX** et **PyTorch** : une même fonction s'applique à un grand nombre d'éléments en parallèle.
VRAM	Mémoire embarquée sur la carte graphique, distincte de la mémoire de l'ordinateur.

Table des matières

Lexique **1**

Présentation du rapport **5**

I Livrable 1 : Reformulation du sujet **6**

1 Contexte et motivation **7**

2 Présentation du sujet **8**
2.1 Énoncé et objectif 8
2.2 Le modèle de Hodgkin-Huxley 8
2.3 Problématique 8

II Livrable 2 : Note de clarification **10**

3 Objet de la note **11**

4 Contexte et besoin **12**
4.1 Contexte 12
4.2 Définition du projet et vision 12
4.3 Besoin exprimé 12
4.4 Données d'entrée 12
4.5 Bête à cornes 13

5 Client et parties prenantes **14**

6 Objectifs **15**
6.1 Livrables 15

7 Périmètre **16**
7.1 Ce qui est inclus 16
7.2 Ce qui est explicitement exclu 16

8 Contraintes, hypothèses et risques **17**
8.1 Contraintes 17
8.2 Hypothèses de travail 17
8.3 Risques 17

9 Critères de réussite **18**

10 Suivi et validation de la note **19**
10.1 Décision : GO / NO GO 19

III Livrable 3 : État de l'art sous forme de benchmark **20**

11 Introduction **21**

12 Fiches d'analyse des articles **22**
12.1 Scalable Parallel Programming with CUDA (2008) . . . 22
12.2 GeNN : a code generation framework for accelerated brain simulations (2016) . . . 23
12.3 Brian2CUDA : Flexible and Efficient Simulation of Spiking Neural Network Models on GPUs (2022) . . . 23
12.4 Automated Translation and Accelerated Solving of Differential Equations on Multiple GPU Platforms (2024) . . . 24
12.5 SimHH : A Versatile, Multi-GPU Simulator for Extended Hodgkin-Huxley Networks (2025) 24

13 Benchmark comparatif **25**

IV Livrable 4 : Analyse fonctionnelle **26**

14 Introduction et expression du besoin **27**
14.1 Bête à cornes . . . 27

15 Fonctions du système **28**
15.1 Liste des fonctions . . . 28
15.2 Diagramme pieuvre . . . 28

16 Caractérisation des fonctions **30**
16.1 Méthode . . . 30
16.2 Tableau de caractérisation . . . 30

17 Hiérarchisation et fonctions cœur du POC **32**
17.1 Méthode du tri croisé . . . 32
17.2 Résultats . . . 32

18 Conclusion et perspectives **34**

V Livrable 5 : Rapport de conception **35**

19 Diagrammes FAST **36**

20 Schéma synoptique **37**

Bibliographie **38**

VI Retour personnel **39**

21 Da Silva Tristan **40**

22 Battaglia Charles-Antoine **41**

23 Botté-Magalhaes Jules **42**

24 Gauthier Keanu **43**

25 Fraysse Ianis **44**

Présentation du rapport

Ce rapport réunit, dans l'ordre du projet, les cinq livrables de la phase d'études : la reformulation du sujet (partie I), la note de clarification (partie II), l'état de l'art sous forme de benchmark (partie III), l'analyse fonctionnelle (partie IV) et le rapport de conception (partie V). Il se termine par le retour personnel de chaque membre du groupe.

Première partie

Livrable 1 : Reformulation du sujet

Contexte et motivation

Ce projet s'inscrit dans le cadre du laboratoire de recherche de M. Antoine Pirog, encadrant du projet R&D au semestre 7 à Junia. Le laboratoire travaille sur la simulation de réseaux de neurones biophysiques, c'est-à-dire des modèles qui reproduisent le comportement électrique de populations de neurones interconnectés.

Le problème auquel ce type de recherche fait face est simple : la simulation de réseaux de neurones réalistes est coûteuse en temps de calcul. Sur **CPU**, un réseau de quelques milliers de neurones peut prendre des heures à simuler. Cela limite la taille des réseaux qu'on peut étudier, et donc les questions scientifiques qu'on peut explorer.

Junia dispose de serveurs équipés de cartes graphiques **RTX 5090**. La question posée est donc : ces **GPU** peuvent-ils apporter la puissance de calcul nécessaire pour rendre la simulation de grands réseaux de neurones tractable au quotidien pour le laboratoire ?

La réponse n'est pas évidente, et elle n'est pas garantie. Le **GPU** est théoriquement beaucoup plus puissant, mais notre problème, un système d'**EDO** couplées avec des connexions irrégulières, n'est pas forcément un cas idéal pour le parallélisme massif. Il est tout à fait possible que la conclusion soit : non, sur ce type de simulation, le **GPU** n'apporte pas un gain suffisant pour justifier le changement d'architecture. Le but du projet est d'apporter une réponse documentée et mesurée à cette question, pas d'arriver à une conclusion prédéterminée.

C'est dans ce contexte que le sujet « Optimisation de modèles neuronaux à oscillateurs couplés » nous a été confié, avec un socle de code déjà existant : modèle de **Hodgkin-Huxley**, **synapses**, solveur **CPU** de référence. On part de cette base pour évaluer l'intérêt du **GPU**.

Présentation du sujet

2.1 Énoncé et objectif

Ce projet a pour objectif d'évaluer la faisabilité de l'utilisation des architectures **GPU** pour la simulation de réseaux de neurones. Il s'agit de déterminer la capacité maximale du système en nombre de **synapses** prises en charge, tout en précisant les conditions dans lesquelles le calcul sur **GPU** surpasse une exécution sur **CPU**. Enfin, les travaux visent à accélérer et à optimiser les performances de simulation en s'appuyant sur nos compétences techniques.

L'approche suivie s'articule en cinq étapes : modélisation **Hodgkin-Huxley**, couplage par **synapses**, construction d'un **connectome**, portage sur **GPU**, puis **benchmark CPU**/**GPU**. L'objectif final est de produire une courbe de **speedup** en fonction de la taille du réseau, qui montre à partir de quel nombre de neurones le **GPU** devient indispensable, et dans quelle mesure il l'est.

2.2 Le modèle de Hodgkin-Huxley

Le socle de code fourni repose sur le modèle de **Hodgkin-Huxley**. Il s'agit d'une modélisation biologique dans laquelle un neurone est assimilé à un circuit électrique, simulant l'activité cérébrale à partir du flux d'ions sodium et potassium. Le modèle décrit l'évolution du **potentiel membranaire** par un système d'équations différentielles ordinaires (**EDO**) gouvernant l'ouverture et la fermeture des **canaux ioniques**. C'est ce formalisme, une **EDO** par neurone répétée sur l'ensemble du réseau, qui rend le problème naturellement parallélisable.

2.3 Problématique

Problématique. Comment accélérer la simulation de réseaux de neurones de **Hodgkin-Huxley** couplés lorsque la taille du **connectome** augmente, et quelle stratégie **GPU** offre le meilleur gain par rapport à la référence **CPU**, à résultats équivalents ?

La vraie question du projet. Le laboratoire de M. Pirog peut-il, grâce aux serveurs Junia équipés de **RTX 5090**, simuler des réseaux de neurones assez grands pour être utiles à la recherche, dans un temps acceptable ?

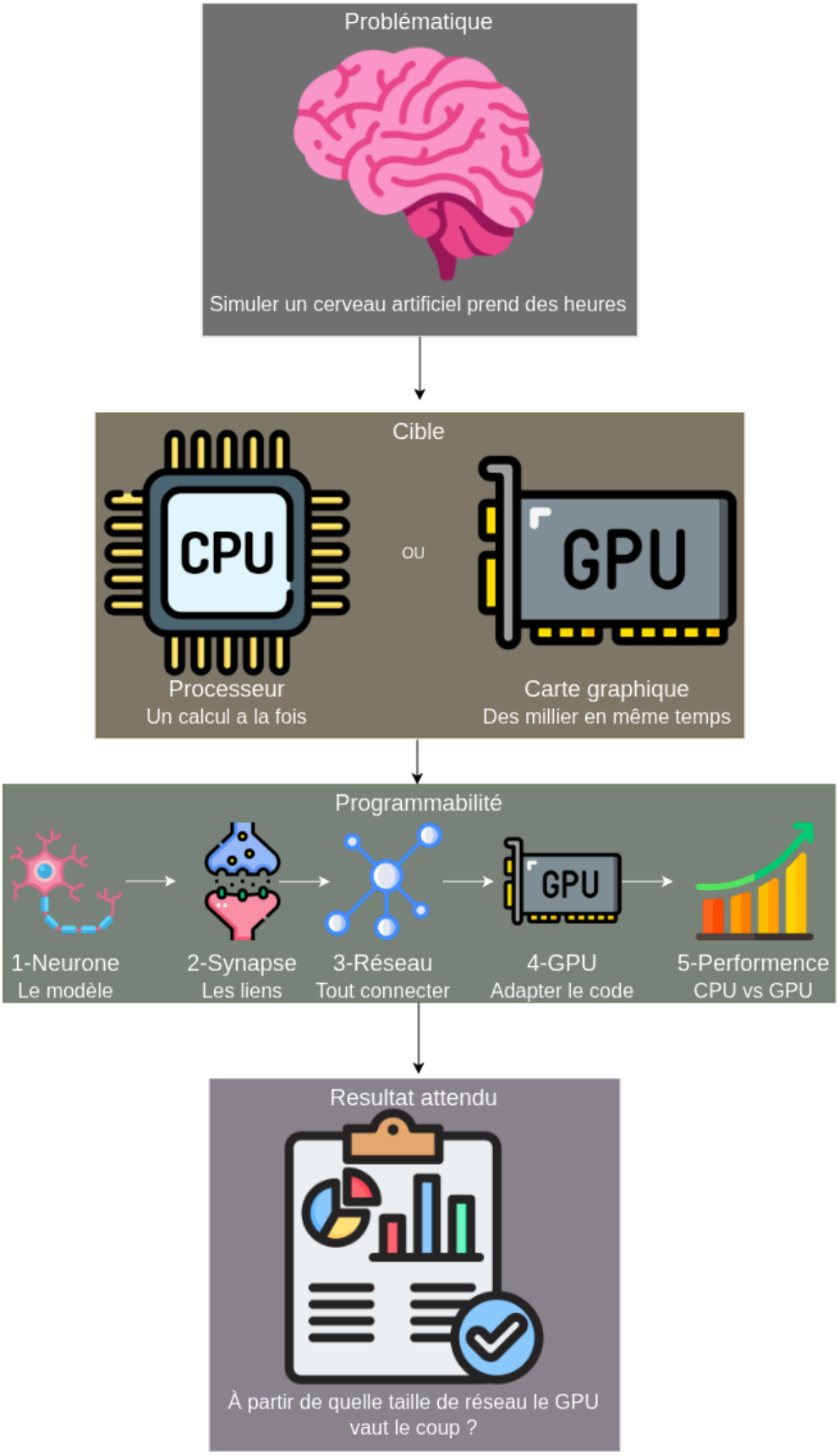


FIGURE 2.1 – Carte conceptuelle du projet : qui, quoi, comment, pourquoi, enjeux et contraintes.

Deuxième partie

Livrable 2 : Note de clarification

Objet de la note

Ce document est le livrable L2 du projet R&D : la **note de clarification**. Le livrable L1 a reformulé le sujet et posé la carte conceptuelle, autrement dit *ce qu'est* le sujet. Avant d'entrer dans l'état de l'art et l'analyse fonctionnelle, il reste à préciser *ce que le projet doit produire, pour qui, dans quel cadre et jusqu'où il va* : c'est le rôle de la présente note.

Le cadrage prend en **entrée** un client porteur de macro-besoins, le laboratoire de M. Pirog, et un existant, le socle de code **CPU** fourni. En **sortie**, il doit donner le contexte et la définition du projet, la vision et les objectifs, les livrables, les étapes clés et le macro-planning, les acteurs et leurs rôles, enfin les contraintes et les risques. Cette note rédige cette synthèse : elle vaut « contrat » avec le client et sert de base au **GO / NO GO** sur la vision commune du projet.

Trois objectifs la guident :

— **écrire noir sur blanc le besoin** du client et des parties prenantes tel que nous l'avons compris, pour que les encadrants le valident ou le corrigent ;

— **délimiter le périmètre** du projet : ce qui est inclus, ce qui est exclu explicitement, et sur quelles hypothèses nous travaillons ;

— **poser des critères de réussite mesurables**, qui guideront l'état de l'art (L3), l'analyse fonctionnelle (L4) et la stratégie de réalisation (L5).

Pour la rédiger, nous nous appuyons sur le sujet « Optimisation de modèles neuronaux à oscillateurs couplés », sur le support du projet et sur le socle de code fourni : modèle de **Hodgkin-Huxley**, **synapses** et solveur **CPU** de référence.

Il s'agit aussi d'un document d'**engagement** : numéroté et daté, il fixe par écrit la vision commune du projet et sera remis à jour au fil des avancées et des clarifications apportées par les encadrants.

Contexte et besoin

4.1 Contexte

Le laboratoire de M. Antoine Pirog étudie des réseaux de neurones biophysiques par simulation. Le modèle de **Hodgkin-Huxley** en forme la base : chaque neurone y devient un système d'**EDO** qui décrit son potentiel membranaire et ses canaux ioniques, et les neurones sont reliés par des modèles biophysiques de **synapses**, répartis selon la topologie d'un **connectome**.

La difficulté centrale est identifiée depuis longtemps : le coût d'une simulation augmente avec la taille du réseau, au point de devenir vite rédhibitoire sur **CPU**. Les réseaux que le laboratoire peut étudier s'en trouvent bornés, et avec eux les questions scientifiques qu'il peut aborder. Junia met à disposition des serveurs à **GPU RTX 5090** dont la puissance théorique mérite qu'on évalue un portage. Le projet peut aussi aboutir à une autre conclusion : qu'une architecture différente, par exemple le **FPGA**, performant sur les calculs répétitifs, conviendrait mieux. Cette piste reste ouverte comme alternative au **GPU**.

4.2 Définition du projet et vision

Définition. Le projet doit mesurer, de façon reproductible, le gain de temps de calcul apporté par un portage **GPU** de la simulation de réseaux de neurones **Hodgkin-Huxley** couplés, et déterminer si ce gain suffit à simuler des connectomes nettement plus grands que la référence **CPU**, sans perdre en fidélité numérique.

Vision. À terme, le laboratoire doit disposer d'un **environnement de simulation générique** : le modèle neuronal et le connectome se choisissent par configuration, la simulation tourne sur **CPU** ou sur **GPU**, et un benchmark documenté dit dans quels cas le **GPU** se justifie. Une réponse négative aurait la même valeur scientifique : elle épargnerait au laboratoire un investissement inutile.

4.3 Besoin exprimé

Le besoin du laboratoire se formule ainsi :

> *Le laboratoire a besoin de pouvoir simuler des réseaux de neurones biophysiques de grandes tailles dans un temps de calcul acceptable, afin d'étudier des connectomes assez grands pour être utiles à la recherche, et de savoir si l'architecture **GPU** dont il dispose apporte un gain réel face à la référence **CPU**.*

Deux points précisent ce besoin :

— l'objectif n'est pas de passer sur **GPU** à tout prix, mais de savoir si le **GPU** se justifie et à quelles conditions : une conclusion négative constituerait, en soi, un résultat ;

— les résultats obtenus sur **GPU** resteront comparables à ceux de la référence **CPU** : pas d'accélération au prix d'une dérive numérique par rapport au modèle de référence.

4.4 Données d'entrée

Le projet démarre avec les éléments suivants, fournis ou imposés :

— **Le sujet** : « Optimisation de modèles neuronaux à oscillateurs couplés ».

— **Le socle de code fourni** : modèle de **Hodgkin-Huxley**, synapses et solveur **CPU**.

— **Les connectomes** : générés artificiellement avec une structure, rangés dans une matrice de taille $N \times N$; aucune donnée biologique réelle n'entre dans le projet.

— **Le benchmark CPU existant** : limité pour l'instant à la topologie en chaîne et à de petites tailles ($N \in \{2, 4, 8, 16\}$), très en dessous des tailles visées ; il servira de point de départ avant d'être étendu.

— **Le matériel** : serveurs Junia à **GPU RTX 5090**.

— **Le calendrier du projet** : jalons et évaluations imposés (voir la section 6.1).

4.5 Bête à cornes

La bête à cornes ci-dessous formalise ce besoin : le système rend service au laboratoire, dans le but d'étudier, en un temps de calcul acceptable, des réseaux de neurones assez grands pour la recherche, en agissant sur les modèles de neurones couplés et les connectomes donnés en entrée.

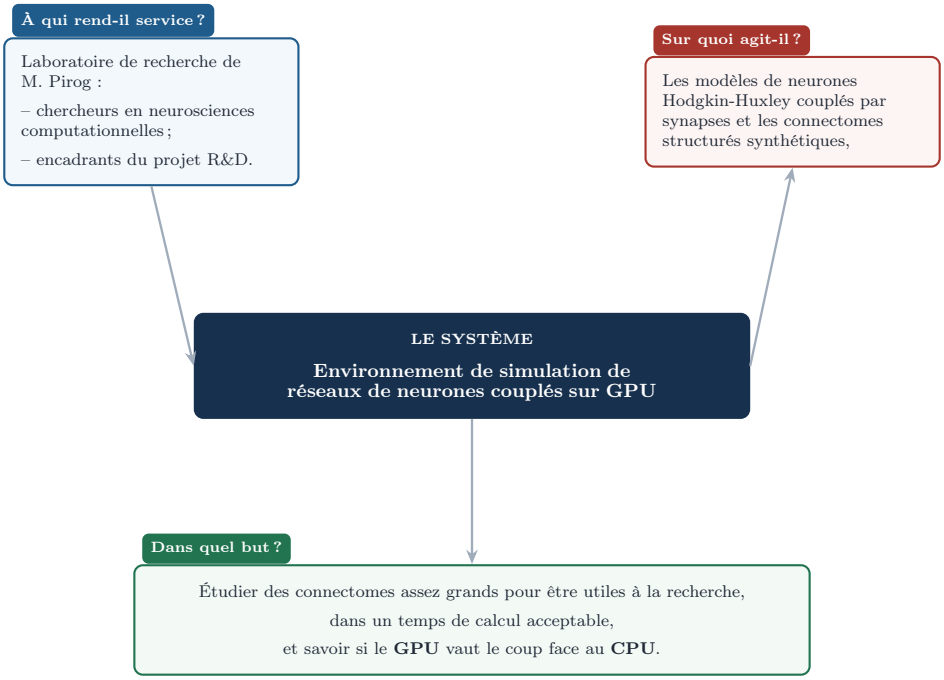


FIGURE 4.1 – Bête à cornes du projet : à qui rend-il service, sur quoi agit-il, dans quel but ?

Client et parties prenantes

Partie prenante	**Rôle**
M. Antoine Pirog	Client principal, encadrant
M. Frédéric Chatrie	Encadrant
Laboratoire de recherche	Utilisateur final
Étudiants du projet R&D	Réalisateurs

Objectifs

Les six attentes du sujet deviennent ici des objectifs opérationnels :

1. **Bâtir une référence fiable** : implémenter le modèle de **Hodgkin-Huxley** avec son terme de couplage synaptique, puis valider sur **CPU** une implémentation de référence qui servira de vérité terrain.
2. **Faire tourner des connectomes structurés** de plus en plus grands : d'abord de petits réseaux de validation, puis des réseaux de plusieurs milliers à dizaines de milliers de neurones.
3. **Développer et comparer des stratégies d'optimisation GPU** de la simulation : parallélisation par neurone, gestion des accès mémoire au connectome.
4. **Mesurer comment le temps de calcul évolue** avec la taille du connectome, sur **CPU** comme sur **GPU**.
5. **Livrer un benchmark comparatif** des approches et des stratégies d'optimisation testées.
6. **Concevoir un environnement générique** où le modèle neuronal et le connectome se remplacent pour d'autres applications sans réécrire la chaîne de simulation.

6.1 Livrables

Les six livrables se font au premier semestre :

— **L1 – Reformulation du sujet avec schéma expliqué**.
— **L2 – Note de clarification** (le présent document).
— **L3 – État de l'art sous forme de benchmark**.
— **L4 – Analyse fonctionnelle**.
— **L5 – FAST et schéma synoptique**.
— **Rapport final et soutenance**.

Le travail après ces livrables se déroule au second semestre, dans la limite des 100 h de réalisation. Il suit trois phases, chiffrées en durées indicatives :

— première phase de développement : environ 2 mois, soit autour de 60 h ;
— phase de tests et de benchmark : environ 2 semaines, soit autour de 15 h ;
— interprétation des résultats puis deuxième phase de développement : environ 1 mois, soit autour de 25 h.

Périmètre

7.1 Ce qui est inclus

— Le modèle de **Hodgkin-Huxley** et le couplage synaptique du socle de code fourni.
— Des connectomes artificiels à structure (topologies régulières ou aléatoires), par tailles croissantes.
— Un code de référence pour **CPU**.
— Les nombreux documents et ressources existants sur le modèle de **Hodgkin-Huxley** sur **CPU**.

7.2 Ce qui est explicitement exclu

— Toute simulation sur données biologiques réelles (connectomes mesurés) : seuls des connectomes synthétiques sont utilisés.
— La modification du modèle biophysique : il ne s'agit pas de chercher un nouveau modèle neuronal, mais d'optimiser la simulation du modèle fourni.
— Le temps réel et l'interaction homme-machine pendant la simulation.

Contraintes, hypothèses et risques

8.1 Contraintes

— **Temps** : 32 h encadrées au S7 pour les études, puis 100 h au S8 pour la réalisation ; le calendrier du projet impose ses jalons (évaluations du 08/10 et du 22/10).

— **Matériel** : des serveurs Junia à **RTX 5090** ; la mémoire du **GPU**, 32 Go de **VRAM**, borne physiquement la taille des connectomes simulables.

— **Couplage irrégulier** : les connexions du connectome n'ont pas de régularité, les accès mémoire ne sont pas uniformes et une partie du calcul reste séquentielle, les pas de temps s'enchaînant l'un après l'autre ; le parallélisme ne sera donc ni parfait ni simple.

— **Techniques** : des langages dictés par l'existant et par le **GPU** : socle **Python** pour la référence, puis **C/C++/CUDA** ou un framework **Python** pour l'accélération et la parallélisation.

8.2 Hypothèses de travail

— Le socle fourni (modèle, synapses, solveur **CPU**) est juste et peut servir de référence sans revalider en profondeur le modèle biophysique ; le portage **GPU** se fera dans un langage comme **C++** ou **CUDA**.

— Les temps mesurés sur les connectomes synthétiques représentent bien ce qu'on obtiendrait sur d'autres connectomes de même structure.

— Le benchmark commencera par les pires cas, les connectomes les plus denses (tout-à-tout), puis comparera avec des connectomes creux, plus légers : l'écart entre les deux mesurera l'effet de la densité de connexions sur les performances.

8.3 Risques

Risque	**Impact**
Mémoire du **GPU** trop juste pour les plus grands connectomes	La taille des réseaux mesurables plafonne
Gain exagéré par une référence **CPU** non optimisée	Conclusion erronée pour le laboratoire

Critères de réussite

Objectif	**Critère de réussite mesurable**
Portage **GPU** valide *ou alternative*	Les sorties du portage **GPU** collent à la référence **CPU** à une tolérance numérique près, définie et justifiée.
Mesures de performance	Courbes du temps de calcul selon la taille du connectome, sur **CPU** et **GPU**, sur une gamme couvrant au moins deux ordres de grandeur.
Benchmark comparatif	Tableau des stratégies testées avec les mêmes métriques (temps, speedup) et un protocole décrit et reproductible.
Environnement générique	Le connectome ou le modèle se remplace par configuration, sans toucher au cœur de la chaîne de simulation.

Suivi et validation de la note

10.1 Décision : GO / NO GO

La validation de cette note vaut **GO** : les parties s'accordent sur le besoin, les objectifs, le périmètre et les durées décrits, et le projet se poursuit sur cette base. Un **GO sous réserve** n'engage la suite qu'une fois les réserves levées ; un **NO GO** signale un désaccord à lever : la note est alors corrigée et soumise à nouveau, chaque modification étant tracée dans l'historique des versions.

Partie	GO	GO sous réserve	NO GO	Observations
Équipe projet	✓	☐	☐	
Client (M. Pirog)	✓	☐	☐	

Date de la décision : 02/10/2026

Troisième partie

Livrable 3 : État de l'art sous forme de benchmark

Introduction

Ce document est le livrable L3 du projet : l'état de l'art sous forme de benchmark. La note de clarification (L2) a posé le besoin : savoir si les serveurs **GPU RTX 5090** de Junia permettent de simuler des réseaux de neurones de **Hodgkin-Huxley** couplés nettement plus grands que sur **CPU**, dans un temps acceptable et sans dérive numérique. Ce document cherche ce que la littérature dit déjà de cette question, pour préparer nos propres mesures.

Nous avons retenu cinq travaux parus entre 2008 et 2025, cherchés dans des sources reconnues : **IEEE Xplore**, PubMed Central, ScienceDirect, l'**ACM** Digital Library et arXiv. Les critères de sélection : simuler des réseaux de neurones (de préférence à conductance, comme Hodgkin-Huxley) ou des systèmes d'**EDO** sur **GPU**, avec des mesures de performance publiées. Chacun fait l'objet d'une fiche d'analyse à six questions (chapitre 12) ; leur synthèse donne le benchmark comparatif du chapitre 13.

D'autres sources complètent cette analyse sans faire l'objet d'une fiche : la fiche technique de la **RTX 5090**, notre matériel cible [9] ; une simulation de réseaux fortement connectés sur **GPU**, notre pire cas [6] ; la parallélisation sur **GPU** du solveur des équations d'un neurone multi-compartiments [7] ; et la multiplication de matrices creuses sur **GPU** [8], la forme sous laquelle notre connectome sera stocké. Toutes sont citées en bibliographie.

Fiches d'analyse des articles

12.1 Scalable Parallel Programming with CUDA (2008)

Cet article fondateur, écrit par des ingénieurs **NVIDIA** et un universitaire [1], explique le modèle de programmation **CUDA** au moment où les **CPU** deviennent multicœurs et les **GPU** massivement parallèles.

Pourquoi ?	Les processeurs grand public gagnent désormais en nombre de cœurs plutôt qu'en fréquence : le matériel devient parallèle, et les logiciels classiques ne le sont pas. Le problème posé : écrire des programmes qui passent à l'échelle quel que soit le nombre de cœurs.
Comment ?	Le modèle **CUDA** repose sur trois abstractions : une hiérarchie de groupes de fils d'exécution, des mémoires partagées et des barrières de synchronisation. Le programmeur répartit le travail en blocs de fils sans connaître le nombre de cœurs : le matériel s'occupe de l'ordonnancement.
Avec quoi ?	Cartes **GPU NVIDIA** et kit de développement **CUDA**, illustrés par des exemples d'applications scientifiques parallèles.
Quels résultats ?	Un même programme **CUDA** s'exécute sur des **GPU** de quelques dizaines ou de plusieurs centaines de cœurs sans être réécrit : le parallélisme suit le matériel.
Quelles limites ?	Article de présentation de 2008 : matériel et outils datés, pas de mesure sur des **EDO** biologiques, et le programmeur travaille toujours à un niveau bas (gestion fine de la mémoire et des fils).
Pour notre sujet ?	C'est le socle conceptuel du projet : paralléliser Hodgkin-Huxley voudra dire répartir les neurones sur des fils d'exécution et organiser la mémoire partagée pour le couplage synaptique.

12.2 GeNN : a code generation framework for accelerated brain simulations (2016)

GeNN (*GPU-enhanced Neuronal Networks*) est une bibliothèque open source développée à l'université de Sussex pour mettre les **GPU** à portée des neuroscientifiques [2].

Pourquoi ?	Simuler des réseaux de neurones réalistes est lent, et les accélérateurs **GPU** restent inaccessibles à ceux qui ne savent pas programmer à bas niveau.
Comment ?	L'utilisateur décrit ses neurones et ses synapses dans une interface de haut niveau ; GeNN génère automatiquement le code **CUDA** correspondant, sans exiger de connaissances **GPU** approfondies.
Avec quoi ?	**GPU NVIDIA** ; bancs d'essai sur plusieurs modèles de réseaux, dont un réseau d'un million de neurones de **Hodgkin-Huxley** à conductance.
Quels résultats ?	Jusqu'à 200 fois plus rapide qu'un cœur de **CPU** pour le réseau d'un million de neurones **Hodgkin-Huxley** ; mais le gain change selon le modèle simulé.
Quelles limites ?	Gain très dépendant du modèle ; comparaison faite contre un seul cœur de **CPU**, donc une référence faible ; **NVIDIA** uniquement.
Pour notre sujet ?	Preuve directe qu'un réseau de **Hodgkin-Huxley** à conductance passe à l'échelle sur **GPU**, et première forme de l'idée de génération de code que notre projet pourra réutiliser ou comparer à des noyaux écrits à la main.

12.3 Brian2CUDA : Flexible and Efficient Simulation of Spiking Neural Network Models on GPUs (2022)

Brian2CUDA ajoute un backend **GPU** au simulateur Brian2, très utilisé en neurosciences computationnelles [3].

Pourquoi ?	Les chercheurs décrivent leurs modèles en **Python** avec le simulateur Brian2, mais celui-ci n'avait pas de sortie **GPU** complète : écrire le code bas niveau soi-même demandait un savoir-faire rare.
Comment ?	Le simulateur génère lui-même le code **CUDA** à partir de la définition du modèle en **Python** : un volet pour l'intégration numérique des états des neurones, un autre pour la propagation des événements synaptiques.
Avec quoi ?	Code **CUDA** généré ; comparaisons avec le backend **CPU** de Brian sur plusieurs types de modèles, y compris neurones et synapses arbitraires et plasticité.
Quels résultats ?	Jusqu'à trois ordres de grandeur d'accélération face au backend **CPU** de Brian ; seule solution **GPU** qui couvre toutes les fonctionnalités de Brian2.
Quelles limites ?	Lié au cadre Brian2 : on doit écrire ses modèles dans ce simulateur ; gain variable selon les modèles ; un seul **GPU**.
Pour notre sujet ?	Démontre qu'on peut accélérer sans écrire de **CUDA** à la main : le modèle décrit en **Python** est traduit. Notre socle **Python**/SciPy pourrait suivre la même voie : compiler les équations plutôt que réécrire le solveur.

12.4 Automated Translation and Accelerated Solving of Differential Equations on Multiple GPU Platforms (2024)

Cet article, publié dans Computer Methods in Applied Mechanics and Engineering [4], cherche à résoudre beaucoup d'**EDO** sur **GPU** sans écrire de code compliqué, et quel que soit le constructeur de la carte.

Pourquoi ?	Résoudre beaucoup d'**EDO** sur **GPU** oblige normalement à écrire du code **CUDA** à la main, et ce code ne fonctionne que sur les cartes **NVIDIA**.
Comment ?	Le solveur d'**EDO** du langage **Julia** traduit tout seul le code pour la carte : on écrit ses équations comme d'habitude, et elles s'exécutent sur le **GPU** sans rien changer au modèle.
Avec quoi ?	Des **GPU** de quatre constructeurs (**NVIDIA**, AMD, Intel, Apple), et des comparaisons avec du **CUDA-C++** écrit à la main puis avec **JAX** et **PyTorch**.
Quels résultats ?	Aussi rapide que du **CUDA-C++** optimisé à la main, et 20 à 100 fois plus rapide que la solution de **JAX** et **PyTorch**. Le même code tourne sur toutes les cartes testées.
Quelles limites ?	Les **EDO** de l'article sont indépendantes les unes des autres : pas de synapses ni de connectome, donc pas de couplage irrégulier comme dans notre cas. Et il faut travailler en **Julia**.
Pour notre sujet ?	Notre réseau est justement un gros système d'**EDO**. L'article montre qu'on peut éviter d'écrire le **CUDA** nous-mêmes et ne pas dépendre d'un seul constructeur. Reste à vérifier si ça marche aussi bien quand les équations sont couplées entre elles.

12.5 SimHH : A Versatile, Multi-GPU Simulator for Extended Hodgkin-Huxley Networks (2025)

SimHH est le travail le plus récent et le plus proche de notre cas : un simulateur de réseaux de **Hodgkin-Huxley** étendus, distribué sur des clusters de calcul [5].

Pourquoi ?	Les réseaux de **Hodgkin-Huxley** étendus (multi-compartiments, jonctions communicantes) dépassent ce qu'une seule machine peut simuler en un temps raisonnable.
Comment ?	Répartition de la simulation sur plusieurs nœuds de calcul avec **MPI** (OpenMPI), avec deux configurations : partager tous les potentiels entre nœuds, ou ne partager que les potentiels utiles ; **CUDA** et **NVLink** pour le calcul, les noyaux tournant en parallèle des communications.
Avec quoi ?	Clusters multi-**GPU** allant jusqu'à 256 nœuds ; modèles étendus dont le noyau inféro-olivaire ; comparaisons avec des **CPU**, un **FPGA** et le simulateur **NEST**.
Quels résultats ?	Réseaux de plus de 10 millions de cellules ; environ 150× plus rapide qu'un **CPU** mono-fil, 10× qu'un **CPU** à 128 fils, 10× qu'un **FPGA**, et 7× que **NEST** sur 32 nœuds pour un réseau de 94 720 neurones à jonctions communicantes.
Quelles limites ?	Exige un cluster de calcul (plusieurs machines reliées) et **MPI** : une infrastructure bien au-delà d'un serveur universitaire ; la configuration de répartition est délicate à régler.
Pour notre sujet ?	État de l'art le plus proche : **Hodgkin-Huxley**, comparaisons honnêtes (**CPU** multi-fils, **FPGA**, simulateurs) et preuve qu'un seul **GPU** finit par saturer. Nos mesures se placeront juste avant cette étape multi-**GPU**.

Benchmark comparatif

Le tableau ci-dessous compare les cinq approches sur les critères de benchmarking. Les gains de performance sont indiqués par rapport à la référence mesurée dans chaque article.

Outil	**Type**	**Technicité**	**Implantation**	**Coût**	**Facilité**	**Gains de performance**	**Moy.**
CUDA [1]	Modèle de programmation **GPU**	Très avancé	Standard en production	Gratuit	Complexe	Sans mesure	+
GeNN [2]	Génération de code **CUDA**	Avancé	Utilisé en recherche	Gratuit	Assez facile	×200 vs 1 cœur **CPU**	+
Brian2CUDA [3]	Backend **GPU** de Brian2	Modéré	Recherche, récent	Gratuit	Très facile	×1000 vs Brian **CPU**	+
Solveur **Julia** [4]	Solveur d'**EDO** multi-**GPU**	Avancé	Écosystème actif	Gratuit	Facile	×20–100 vs **JAX**/ **PyTorch**	+
SimHH [5]	Multi-**GPU** (**MPI**)	Très avancé	Prototype d'une équipe	Exige un cluster	Complexe	×150 vs **CPU** mono-fil	-

Couleur	**Signification**
	++ Meilleur cas
	+ Bon cas
	- Cas plutôt moyen
	− Cas moyen
	Non concerné

Quatrième partie

Livrable 4 : Analyse fonctionnelle

Introduction et expression du besoin

Ce document est le livrable L4 du projet R&D : l'**analyse fonctionnelle**. Le livrable L2 a posé le besoin du laboratoire de M. Pirog et cadré le projet ; le livrable L3 a montré, par l'état de l'art, les ordres de grandeur que le **GPU** peut apporter. Ce document fait le passage suivant : il décrit *ce que* le système doit faire, sans préjuger du *comment*, pour préparer les choix techniques du second semestre. Le besoin est d'abord exprimé et validé ; les fonctions sont ensuite recensées et représentées par le diagramme pieuvre, puis caractérisées et enfin hiérarchisées pour dégager les fonctions cœur du **POC**.

Le système étudié est celui de la vision posée en L2 : un **environnement de simulation de réseaux de neurones couplés sur GPU**. Son besoin, tel que l'a validé la note de clarification, tient en une phrase : *simuler des réseaux de neurones biophysiques de grande taille dans un temps de calcul acceptable, et savoir si le **GPU** apporte un gain réel face à la référence **CPU**.*

14.1 Bête à cornes

La bête à cornes qui exprime ce besoin est celle établie lors de la note de clarification, reprise à la figure 4.1.

Fonctions du système

15.1 Liste des fonctions

Le système doit remplir des fonctions, réparties en trois familles : les fonctions principales (FP), les fonctions secondaires (FS) et les fonctions de contrainte (FC).

Code	Fonction	Éléments reliés
FP1	Simuler des réseaux de neurones **Hodgkin-Huxley** couplés par **synapses**, en calculant l'évolution des potentiels membranaires à chaque pas de temps.	Modèle neuronal, connectome
FP2	Accélérer la simulation sur **GPU**.	Serveurs **GPU**, référence **CPU**
FP3	Restituer les résultats aux chercheurs (tracés, temps de calcul, erreurs) sous une forme lisible et exploitable.	Chercheur, modèle neuronal
FS1	S'adapter à différentes tailles et topologies de connectomes.	Connectome
FS2	Adapter les équations du modèle neuronal pour simuler d'autres phénomènes (autres modèles de neurones, variantes des équations) sans réécrire la chaîne de simulation.	Modèle neuronal
FS3	Garantir des mesures reproductibles : protocole de benchmark documenté, mesures répétées, graines fixées.	Encadrants
FC1	Respecter le modèle de **Hodgkin-Huxley** et sa méthode de résolution : mêmes équations, même schéma d'intégration que le socle de référence.	Référence **CPU**
FC2	Fonctionner sur le matériel cible : un **GPU**.	Serveurs **GPU**
FC3	Respecter le temps et le calendrier du projet (32 h d'études au S7, 100 h de réalisation au S8, jalons imposés).	Calendrier

15.2 Diagramme pieuvre

Le système interagit avec plusieurs éléments de son environnement : le chercheur du laboratoire, le modèle neuronal, le connectome, les serveurs GPU, ou encore la référence CPU. Ces interactions sont traduites par des fonctions principales, secondaires et des contraintes, présentées dans le diagramme pieuvre ci-dessous.

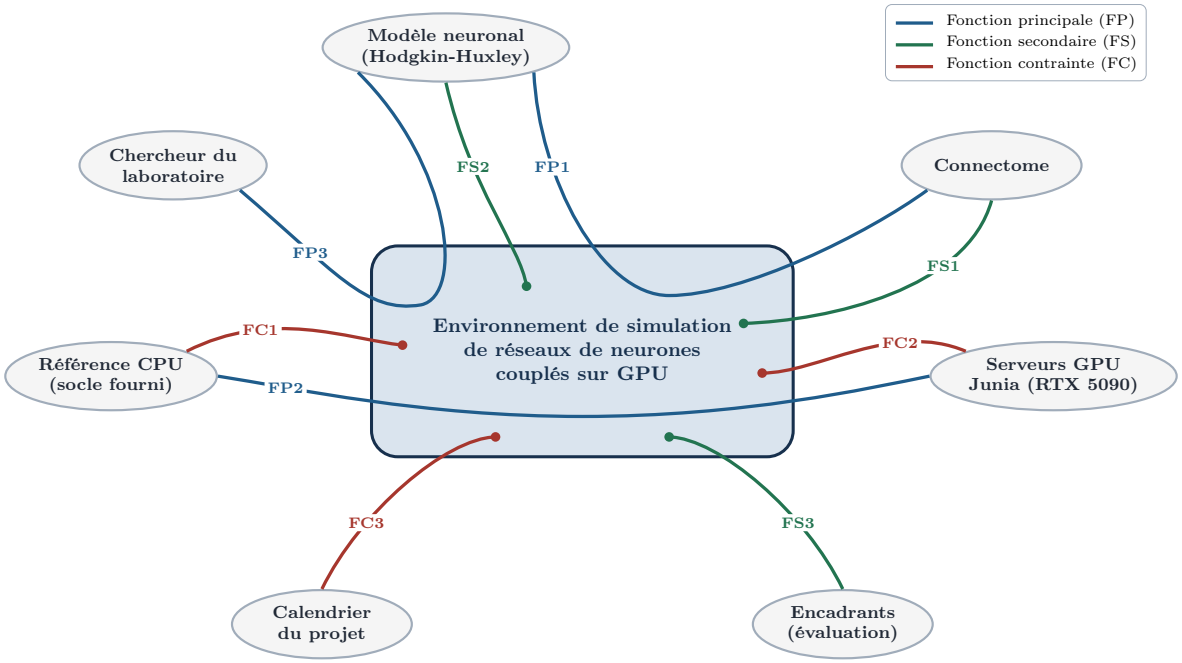


FIGURE 15.1 – Diagramme pieuvre : les fonctions principales traversent le système pour relier deux éléments extérieurs ; les autres s'arrêtent dans le système.

Caractérisation des fonctions

16.1 Méthode

Chaque fonction est caractérisée par un ou plusieurs **critères**, un **niveau** visé pour chacun, une **tolérance** et une **flexibilité**. Les classes de flexibilité sont celles du cahier des charges fonctionnel : **F0** impérative (limite physique ou cadre imposé), **F1** peu négociable, **F2** négociable (un résultat en dessous du niveau reste exploitable) et **F3** très négociable.

16.2 Tableau de caractérisation

Les lignes sont colorées selon la famille de la fonction et les fonctions cœur du **POC** sont repérées par une étoile (*).

Fct.	Critère	Niveau	Tolérance	Flex.
FP1*	Taille du réseau simulé	10^3 neurones	Quelques centaines (100)	F1
	Modèle simulé	Neurone de **Hodgkin-Huxley** et **synapse** du socle fourni	Aucune	F0
FP2*	**Speedup** face à la référence **CPU** du socle	≥ 10 (pour réseau de taille importante)	> 1 (un gain nul reste un résultat)	F2
	Speedup face à une référence **CPU** optimisée (multi-cœur)	Mesuré face au socle seul, limite signalée	Aucune	F1
	Gamme de tailles N et seuil du **GPU**	Taille à partir de laquelle le **GPU** devient plus rapide, identifiée pour chaque topologie	Identifiée pour au moins une topologie	F1
FP3	Résultats restitués	Tracés, temps de calcul, erreurs, fichiers lisibles par **Python** (NumPy, CSV)	Tracés et temps de calcul seuls	F1
	Format des fichiers	Lisible en **Python** (NumPy, CSV)	Tout format ouvert	F2
FS1*	Topologies de connectome	Chaîne, tout-à-tout, aléatoire, connectome creux	Chaîne et tout-à-tout suffisent	F1
FS2	Changement d'équations	Par configuration, sans modifier le moteur	Modification d'un seul fichier ; second modèle si le temps le permet	F1
FS3	Optimisation du temps et protocole	Calcul et transferts mémoire séparés ; protocole de benchmark écrit dans le rapport	Temps de calcul uniquement	F1
FC1*	Équations et schéma d'intégration	Identiques au socle de référence	Aucune	F0
	Précision des calculs	Double précision (FP64), comme la référence	Simple précision (FP32) si les tracés restent superposés à la référence	F2
FC2*	Matériel et mémoire	**RTX 5090** 32 Go de **VRAM**	Aucune	F0

Fct.	Critère	Niveau	Tolérance	Flex.
FC3	Volume horaire et jalons	32 h au S7 ; 100 h au S8 (60 h de développement, 15 h de tests, 25 h d'analyse) ; jalons respectés	Aucune	F0

Légende. ▭ fonction principale (FP) ▭ fonction secondaire (FS) ▭ fonction contrainte (FC) * : fonction cœur du **POC**.

Hiérarchisation et fonctions cœur du POC

17.1 Méthode du tri croisé

Les fonctions sont comparées deux à deux : chaque case de la matrice indique la fonction la plus importante pour le service rendu au laboratoire, et l'écart d'importance va de **0** (égalité, aucun point) à **1** (un peu plus), **2** (plus) et **3** (beaucoup plus).

	FP2	**FP3**	**FS1**	**FS2**	**FS3**	**FC1**	**FC2**	**FC3**
FP1	FP1(1)	FP1(2)	FP1(2)	FP1(3)	FP1(2)	FP1(1)	FP1(1)	FP1(2)
FP2		FP2(2)	FP2(1)	FP2(3)	FP2(1)	0	FP2(1)	FP2(2)
FP3			0	FP3(2)	0	FC1(1)	0	FP3(1)
FS1				FS1(2)	FS1(1)	FC1(1)	0	FS1(2)
FS2					FS3(1)	FC1(3)	FC2(2)	FC3(1)
FS3						FC1(1)	FC2(1)	FS3(1)
FC1							FC1(1)	FC1(2)
FC2								FC2(1)
FC3								

Table 17.1 – Matrice du tri croisé (36 comparaisons).

17.2 Résultats

Fonction	**Points**	**Poids**	**Intitulé**
FP1	14	29,2 %	Simuler des réseaux **Hodgkin-Huxley** couplés par **synapses**
FP2	10	20,8 %	Accélérer la simulation sur **GPU**
FC1	9	18,8 %	Respecter le modèle et le schéma d'intégration du socle
FS1	5	10,4 %	S'adapter à différentes tailles et topologies de connectomes
FC2	4	8,3 %	Fonctionner sur le matériel cible (**GPU**)
FP3	3	6,3 %	Restituer les résultats aux chercheurs
FS3	2	4,2 %	Garantir des mesures reproductibles
FC3	1	2,0 %	Respecter le temps et le calendrier du projet
FS2	0	0,0 %	Adapter les équations sans réécrire la chaîne de simulation
Total	48	100 %	

Table 17.2 – Hiérarchisation des fonctions, par poids décroissant ; la couleur va du vert (poids fort) au rouge (poids nul).

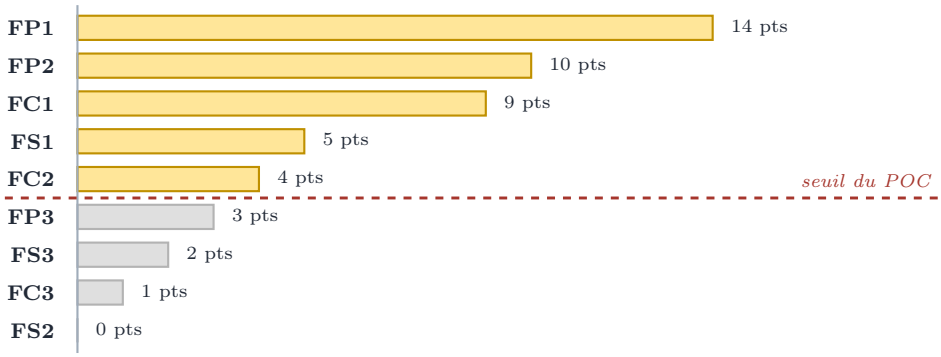


FIGURE 17.1 – Poids des fonctions issus du tri croisé.

Conclusion et perspectives

Ce livrable fait le point sur ce que le système doit faire pour répondre au besoin du laboratoire de M. Pirog : simuler de grands réseaux de neurones dans un temps raisonnable, et savoir si le **GPU** est vraiment plus rapide que le **CPU**. Le besoin a d'abord été rappelé avec la bête à cornes, puis traduit en fonctions principales, secondaires et de contrainte, reliées aux éléments extérieurs dans le diagramme pieuvre.

Chaque fonction a ensuite été décrite précisément : ce qui est attendu (le critère), jusqu'où (le niveau), et la marge de négociation (la flexibilité). Ce travail fixe des objectifs clairs et chiffrés pour la suite du projet, comme la taille des réseaux visés ou le gain de temps attendu.

Enfin, le tri croisé a permis de classer les fonctions par importance. Les cinq premières – FP1, FP2, FC1, FS1 et FC2 – forment les fonctions cœur du **POC** : elles totalisent **87,5 %** des points, et ce sont elles qui permettront de répondre à la question du projet. Les autres fonctions ne sont pas abandonnées : elles seront traitées plus tard, si le temps du semestre 8 le permet.

Cinquième partie

Livrable 5 : Rapport de conception

Diagrammes FAST

Le diagramme **FAST** se lit de gauche à droite avec la question « *comment* ? » ; dans l'autre sens, la même chaîne répond à « *pourquoi* ? ». Chaque sous-fonction est reliée, par une flèche pointillée, à la solution technique (en jaune) qui la réalise. Dans tout le document, W désigne la matrice de connexions du réseau.

Légende. fonction sous-fonction solution technique.

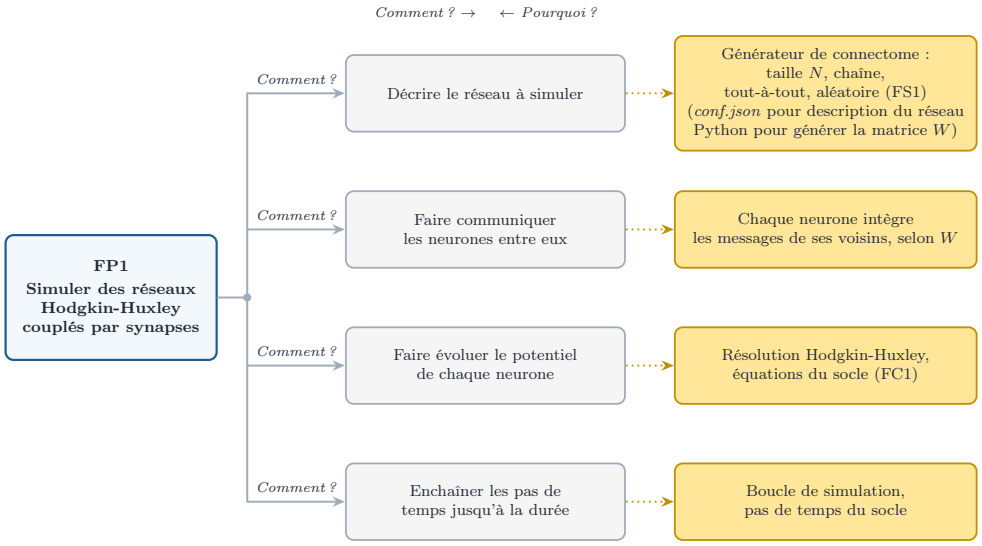


FIGURE 19.1 – FAST de FP1 : le réseau est décrit par le générateur de connectome (FS1), puis simulé pas à pas – communication entre les neurones, mise à jour Hodgkin-Huxley (FC1) – jusqu'à la durée choisie.

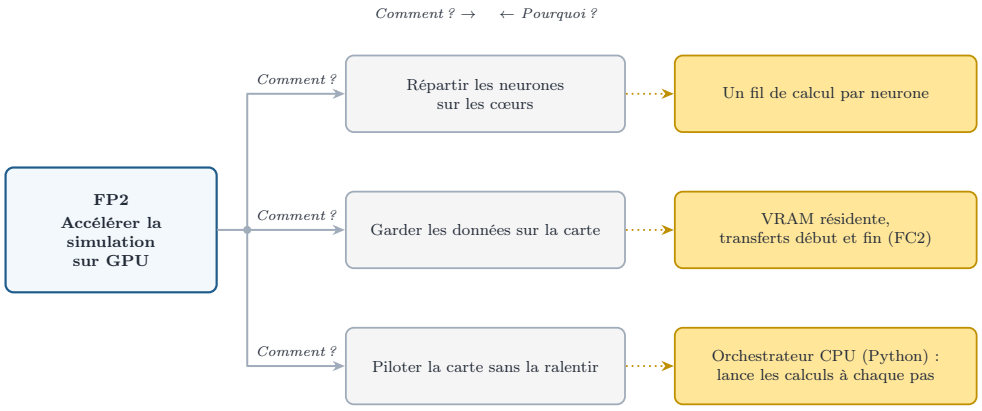


FIGURE 19.2 – FAST de FP2 : les neurones sont répartis sur les cœurs, les données restent en VRAM (FC2) et l'orchestrateur lance les calculs à chaque pas.

Schéma synoptique

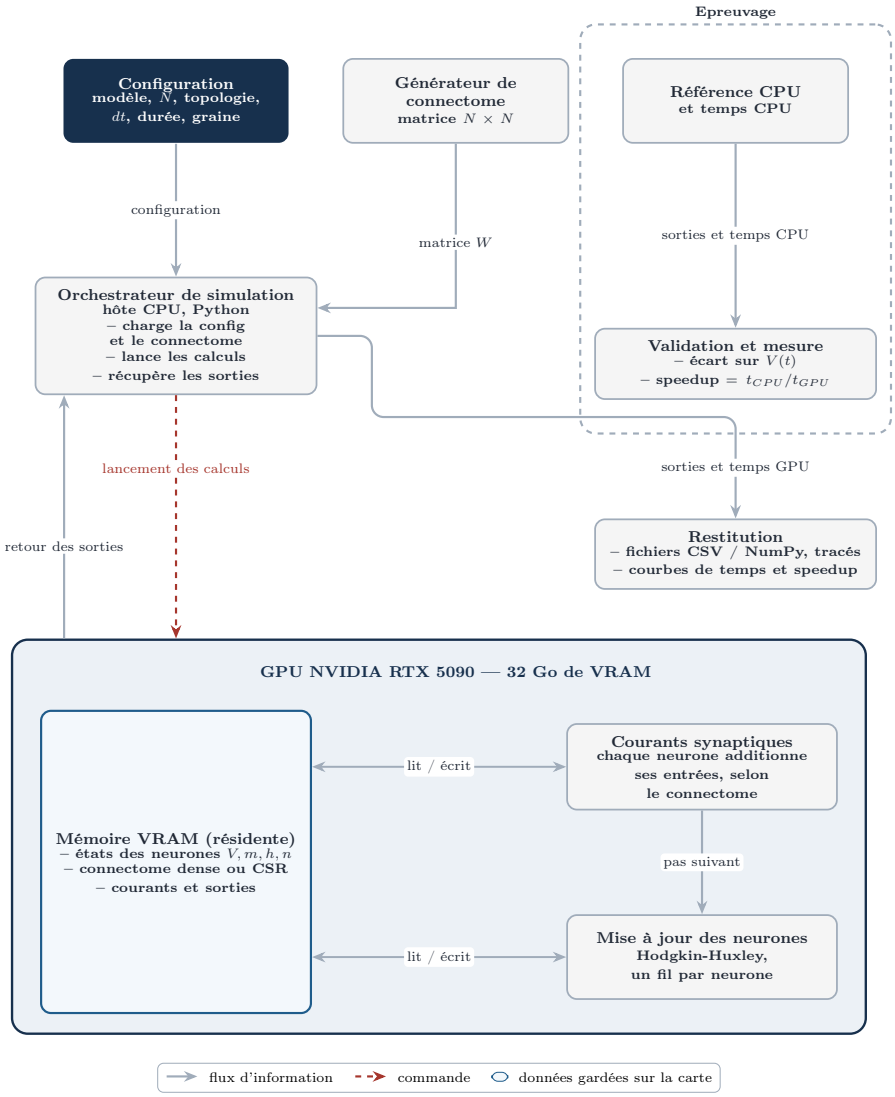


FIGURE 20.1 – Schéma synoptique du POC : l'orchestrateur sur CPU pilote la carte GPU ; la référence CPU et les sorties GPU sont comparées puis restituées.

L'orchestrateur, sur le **CPU** hôte, charge la configuration et le connectome, lance les calculs à chaque pas et récupère les sorties ; pendant ce temps, toute la simulation vit dans la **VRAM** de la carte, sans transfert en cours de calcul. Les fonctions cœur se repèrent bloc par bloc : FP1 dans les deux calculs de la carte (courants synaptiques et **Hodgkin-Huxley**), FP2 dans ces calculs et la mesure du **speedup**, FC1 dans la validation, FS1 dans le générateur et le stockage dense ou **CSR**, FC2 dans la carte visée (**RTX 5090**, 32 Go de **VRAM**).

Bibliographie

[1] J. Nickolls, I. Buck, M. Garland, K. Skadron, « Scalable Parallel Programming with CUDA », *ACM Queue*, vol. 6, n° 2, p. 24–32, mars 2008. DOI : 10.1145/1365490.1365500.

[2] E. Yavuz, J. Turner, T. Nowotny, « GeNN : a code generation framework for accelerated brain simulations », *Scientific Reports*, vol. 6, article 18854, 2016. DOI : 10.1038/srep18854.

[3] D. Alevi, M. Stimberg, H. Sprekeler, K. Obermayer, M. Augustin, « Brian2CUDA : Flexible and Efficient Simulation of Spiking Neural Network Models on GPUs », *Frontiers in Neuroinformatics*, vol. 16, article 883700, 2022. DOI : 10.3389/fninf.2022.883700.

[4] U. Utkarsh, V. Churavy, Y. Ma, T. Besard, P. Srisuma, T. Gymnich, A. R. Gerlach, A. Edelman, G. Barbastathis, R. D. Braatz, C. Rackauckas, « Automated Translation and Accelerated Solving of Differential Equations on Multiple GPU Platforms », *Computer Methods in Applied Mechanics and Engineering*, vol. 419, 2024. DOI : 10.1016/j.cma.2023.116591. Prépublication : arXiv:2304.06835.

[5] M. C. W. Engelen, J.-H. L. F. Betting, C. Strydis, « SimHH : A Versatile, Multi-GPU Simulator for Extended Hodgkin-Huxley Networks », *IEEE Access*, vol. 13, 2025. DOI : 10.1109/ACCESS.2025.3550444.

[6] B. Golosio, G. Tiddia, C. De Luca, E. Pastorelli, F. Simula, P. S. Paolucci, « Fast Simulations of Highly-Connected Spiking Cortical Models Using GPUs », *Frontiers in Computational Neuroscience*, vol. 15, article 627620, 2021. DOI : 10.3389/fncom.2021.627620.

[7] D. T. Vooturi, K. Kothapalli, U. S. Bhalla, « Parallelizing Hines Matrix Solver in Neuron Simulations on GPU », *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*, p. 388–397, 2018. DOI : 10.1109/HiPC.2017.00051.

[8] N. Bell, M. Garland, « Implementing sparse matrix-vector multiplication on throughput-oriented processors », *Proceedings of the Conference on High Performance Computing, Networking, Storage and Analysis (SC'09)*, ACM, 2009. DOI : 10.1145/1654059.1654078.

[9] « NVIDIA GeForce RTX 5090 », fiche technique, TechPowerUp. www.techpowerup.com/gpu-specs/geforce-rtx-5090.c4216.

Sixième partie

Retour personnel

Da Silva Tristan

Ayant déjà travaillé dans l'électronique et l'informatique pour la santé, j'ai choisi ce sujet pour approfondir ces deux domaines, tout en partant de zéro sur les modèles biophysiques de neurosciences comme celui de Hodgkin-Huxley. Au début, la gestion de projet et l'analyse fonctionnelle m'ont semblé assez floues et très théoriques.

Le premier travail de reformulation m'a permis de poser une première représentation du projet et de sortir de l'impression de flou du démarrage. Même si ma compréhension restait encore partielle à ce stade, le travail en équipe a permis de mutualiser nos connaissances et de faciliter l'assimilation du sujet.

La deuxième étape, la note de cadrage, a consisté à mettre au propre les objectifs, le périmètre et les contraintes, ce qui m'a aidé à bien cerner le besoin et à savoir exactement vers quoi nous allions.

J'ai ensuite trouvé la partie sur l'état de l'art et le benchmark très utile, car elle m'a permis de prendre connaissance des technologies existantes et de bien identifier les verrous techniques liés à l'optimisation GPU. C'est l'analyse fonctionnelle qui m'a le plus intéressé : la matrice de tri croisé et la hiérarchisation des fonctions ont permis de mettre en évidence ce qui était le plus urgent, réaliste et prioritaire à intégrer dans notre POC.

Enfin, définir l'architecture technique et le choix des sous-ensembles a été très formateur, car cela a concrétisé la solution et m'a donné une vision claire de ce qu'il reste à implémenter.

Battaglia Charles-Antoine

J'ai choisi ce sujet parce qu'il me semblait proche de l'approche réelle d'un ingénieur de recherche : un besoin concret venu d'un laboratoire, une question à instruire, des mesures à produire. Ce lien direct avec la réalité d'une recherche donnait du sens au travail, et j'avais le sentiment que mes compétences purement informatiques pouvaient avoir un impact au-delà du numérique.

Cette première phase du projet m'a permis d'explorer en profondeur un domaine nouveau pour moi : la simulation de réseaux de neurones Hodgkin-Huxley et l'accélération sur GPU. Autour d'une question simple, « le GPU vaut-il le coup face au CPU ? », j'ai construit pas à pas un vocabulaire métier, du connectome au speedup, en rédigeant les cinq livrables.

J'ai pris progressivement une posture de coordination au sein de l'équipe : répartir les tâches, suivre l'avancement de chacun aux séances, et faire le lien entre les remarques des encadrants et les documents. Une grande part de mon travail a été de la consolidation : garder le même vocabulaire d'un livrable à l'autre, relire les chiffres du tri croisé, harmoniser les schémas.

La principale difficulté a été d'intégrer les retours reçus en cours de route sans casser la cohérence d'ensemble : chaque remarque des encadrants se répercute sur plusieurs livrables à la fois. J'ai appris à écouter, trancher et faire converger le groupe sur une idée commune. Cela permet de gagner en efficacité et de produire un document final plus solide.

Botté-Magalhaes Jules

J'ai choisi ce sujet parce que je souhaite m'orienter vers la Data et l'IA. J'y voyais une occasion de découvrir les réseaux de neurones et d'aborder la simulation ainsi que le calcul sur GPU. J'ai surtout contribué aux recherches bibliographiques, au premier livrable et à la relecture des documents.

Au début, j'ai eu du mal à comprendre les notions biophysiques. Le modèle de Hodgkin-Huxley, les canaux ioniques et les synapses étaient nouveaux pour moi. J'ai dû prendre le temps de comprendre ce vocabulaire pour mieux suivre le projet. L'optimisation sur GPU reste aussi un point que je dois approfondir.

La recherche documentaire m'a demandé beaucoup de tri. Il y avait énormément d'articles et de brevets, mais tous ne répondaient pas à notre sujet. Certains étaient également difficiles à comprendre avec mes connaissances de départ. J'ai appris à mieux cibler mes recherches, même si sélectionner les documents les plus utiles n'était pas toujours évident.

Enfin, les relectures collectives m'ont aidé à mieux comprendre les différentes parties du projet et à repérer les passages qui manquaient de cohérence. Lire les textes des autres m'a aussi fait découvrir d'autres façons de présenter les idées et de structurer un rapport. Pour un prochain projet, je commencerais plus tôt un glossaire et des notes de lecture pour retrouver plus facilement les notions et les sources utiles.

Gauthier Keanu

Mon rôle. On était cinq sur le projet. De mon côté, je me suis surtout occupé de la recherche bibliographique (l'état de l'art du livrable 3) et, dans une moindre mesure, de la coordination : répartir les livrables entre nous et vérifier chaque semaine qu'on tenait les échéances. Événement marquant. Ce qui m'a le plus marqué, c'est le moment où on a compris que le modèle de Hodgkin-Huxley ne servait en fait que de banc d'essai, et que l'encadrant visait des réseaux de cellules pancréatiques. On avait lu le sujet de façon trop littérale au départ. Du coup, la modularité du code n'était plus juste une bonne pratique, c'est devenu une vraie contrainte de travail pour nous. La prochaine fois, je poserai ce genre de questions au commanditaire dès le début au lieu de m'en tenir à l'énoncé. Difficulté rencontrée. Le sujet était dense. Biophysique du neurone, équations différentielles, calcul sur GPU : rien de tout ça ne me était familier avant le projet. Plutôt que de chercher à tout comprendre d'un coup, je suis reparti du code fourni, parce qu'il m'aidait à relier les équations à quelque chose de concret. J'ai aussi lu la littérature par étapes, d'abord des synthèses, puis des articles plus techniques quand j'en avais réellement besoin. Ce que j'en retiens, c'est qu'on n'a pas besoin de tout maîtriser pour avancer. Il faut juste atteindre le niveau demandé par l'étape en cours. Compétences développées. J'ai beaucoup progressé sur la lecture ciblée de sources et sur le fait de vérifier ce qu'on trouve avant de l'utiliser. L'analyse fonctionnelle et la rédaction en LATEX m'ont aussi pris du temps au début, mais ça rentre. Côté équipe, j'ai appris à découper le travail et à formaliser les décisions par écrit, parce qu'à l'oral on oublie facilement la moitié de ce qui a été dit.

Ce que je ferais différemment. Au semestre 8, si c'était à refaire : préparer les questions pour le commanditaire dès la première semaine, mettre en place un planning partagé dès le départ, et passer plus vite au concret avec le code CPU puis un premier noyau GPU. On a perdu du temps à vouloir cadrer avant d'essayer.

Fraysse Ianis

J'ai choisi ce projet parce qu'il me permettait d'apprendre comment on simule numériquement des modèles de neurones : faire reproduire par un ordinateur le comportement électrique d'un réseau de neurones biologiques, à partir d'équations, est un sujet qui m'attirait sans que j'aie eu l'occasion de l'aborder.

J'ai surtout travaillé sur les schémas (carte conceptuelle, bête à cornes, synoptique, avis sur le FAST) et participé à la rédaction et relecture des livrables 1,2, 4, 5. J'ai moins contribué au livrable 3 car j'étais moins à l'aise avec le benchmark d'articles techniques sur le GPU.

En faisant les schémas, j'ai appris qu'on ne peut pas dessiner un schéma sur un sujet qu'on ne comprend pas : la carte conceptuelle a obligé le groupe à se mettre d'accord sur les étapes du projet. Le travail en groupe m'a aussi appris à confronter mon point de vue à celui des autres, à répartir les tâches selon les points forts de chacun et à m'appuyer sur leurs explications quand je bloquais.

J'ai aussi compris que le but n'était pas de passer sur GPU à tout prix, mais de savoir s'il valait le coup.

Ma principale difficulté a été d'entrer dans un sujet qui était assez vaste et loin de mes connaissances ; je l'ai surmontée en partant des idées générales et des explications du groupe.